

MODEL ABSTRACTION LAYER

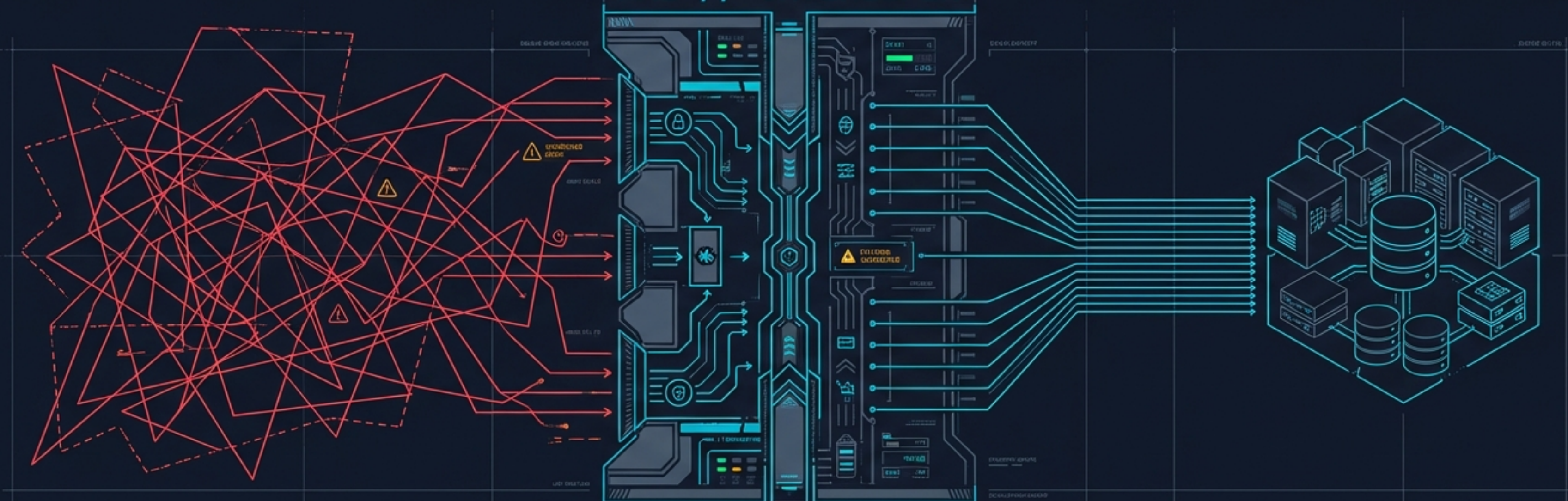
Engineering Safer Agentic Access to Real Systems

A framework for capability attenuation and data minimisation in the era of the Model Context Protocol (MCP). Based on the white paper 'Model Abstraction Layer for Safer Agentic Access to Real Systems'.

UNCHECKED AGENT ACCESS

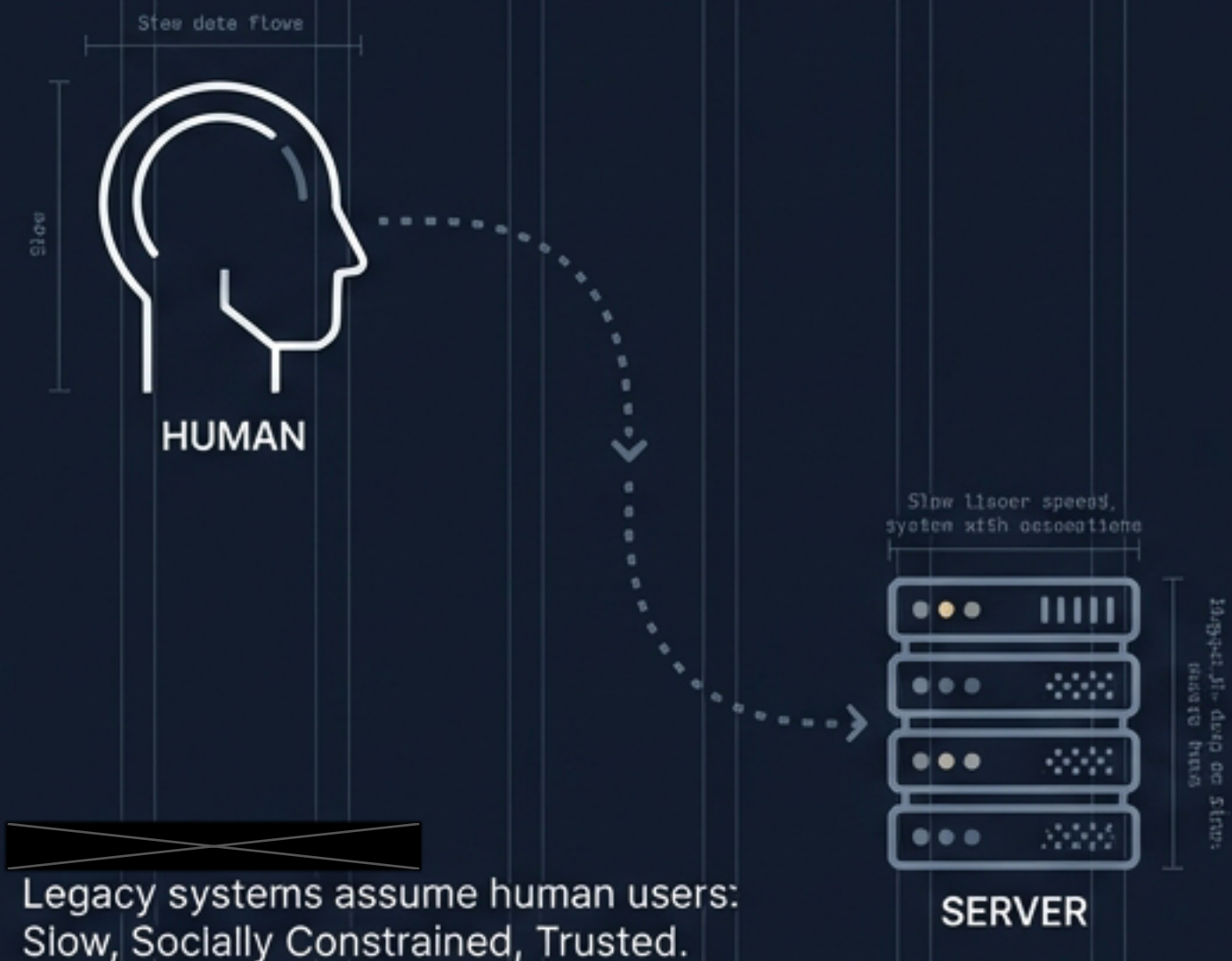
MAL // GATEKEEPER

SECURE REAL SYSTEMS

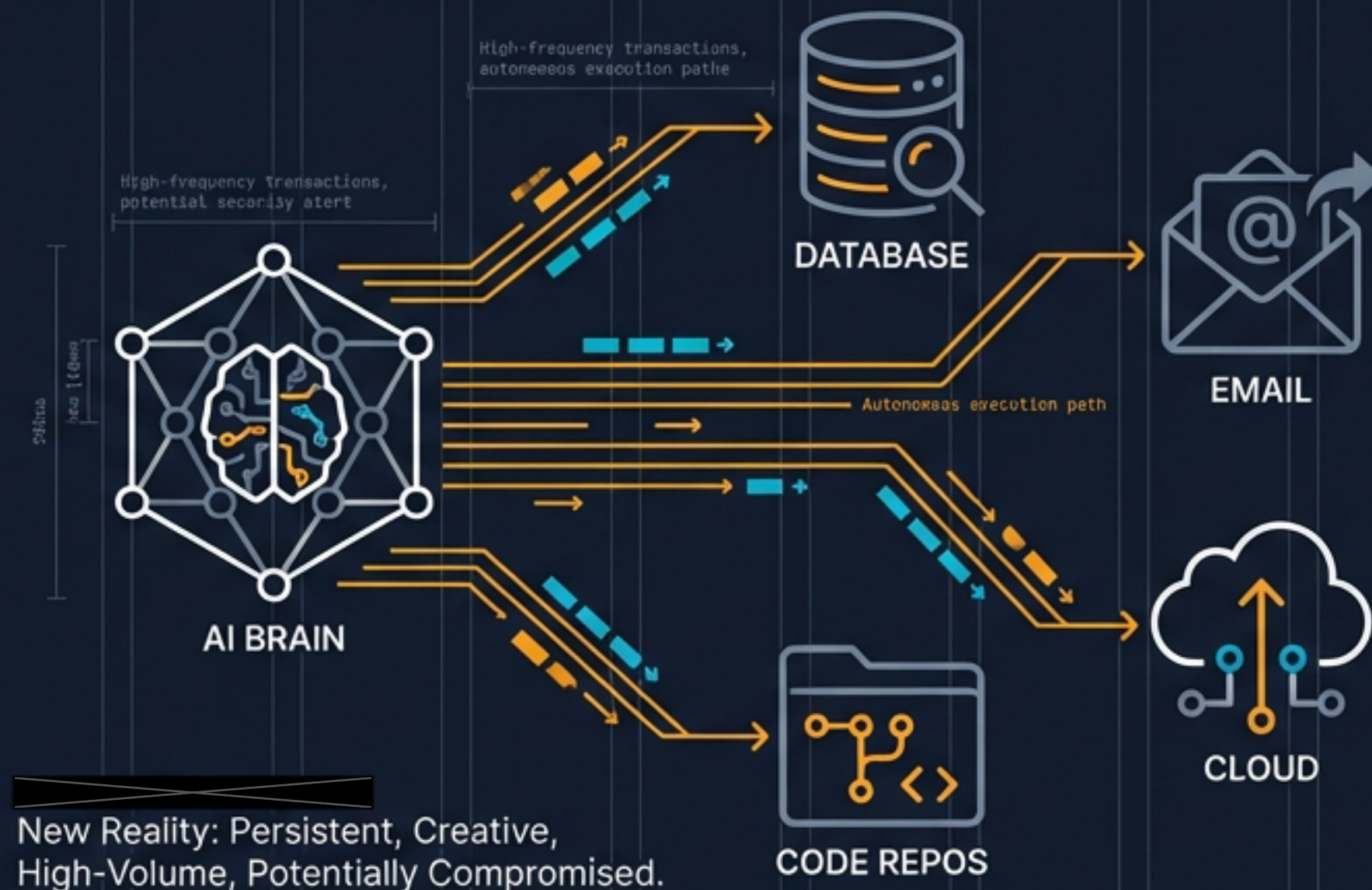


THE SHIFT FROM CHAT TO ACTION CREATES AN INTEGRATION PARADOX

HUMAN-CENTRIC SYSTEMS



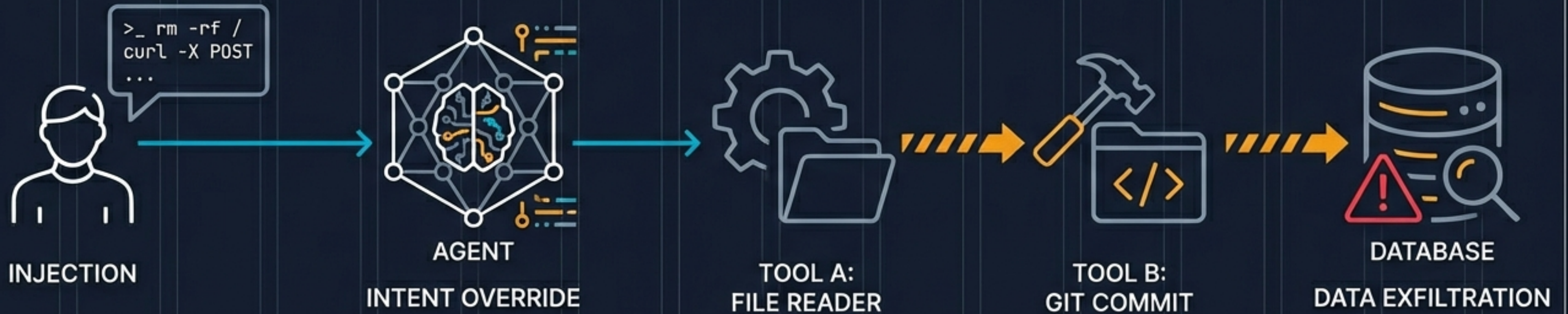
AGENTIC WORKFLOWS



KEY INSIGHT: AGENTIC WORKFLOWS INVERT THE RISK MODEL. WE ARE CONNECTING SYSTEMS DESIGNED FOR TRUSTED HUMANS TO AUTONOMOUS PROCESSES THAT CAN BE TRICKED.

PROMPT INJECTION IS NOW A REMOTE CODE EXECUTION VECTOR

JetBrains Mono Regular
CASE STUDY: 2026 GIT MCP VULNERABILITY (incp-server-git).
ARGUMENT INJECTION AND PATH VALIDATION FAILURES
ENABLED FILE TANPERING.



OWASP LLM01:
PROMPT INJECTION

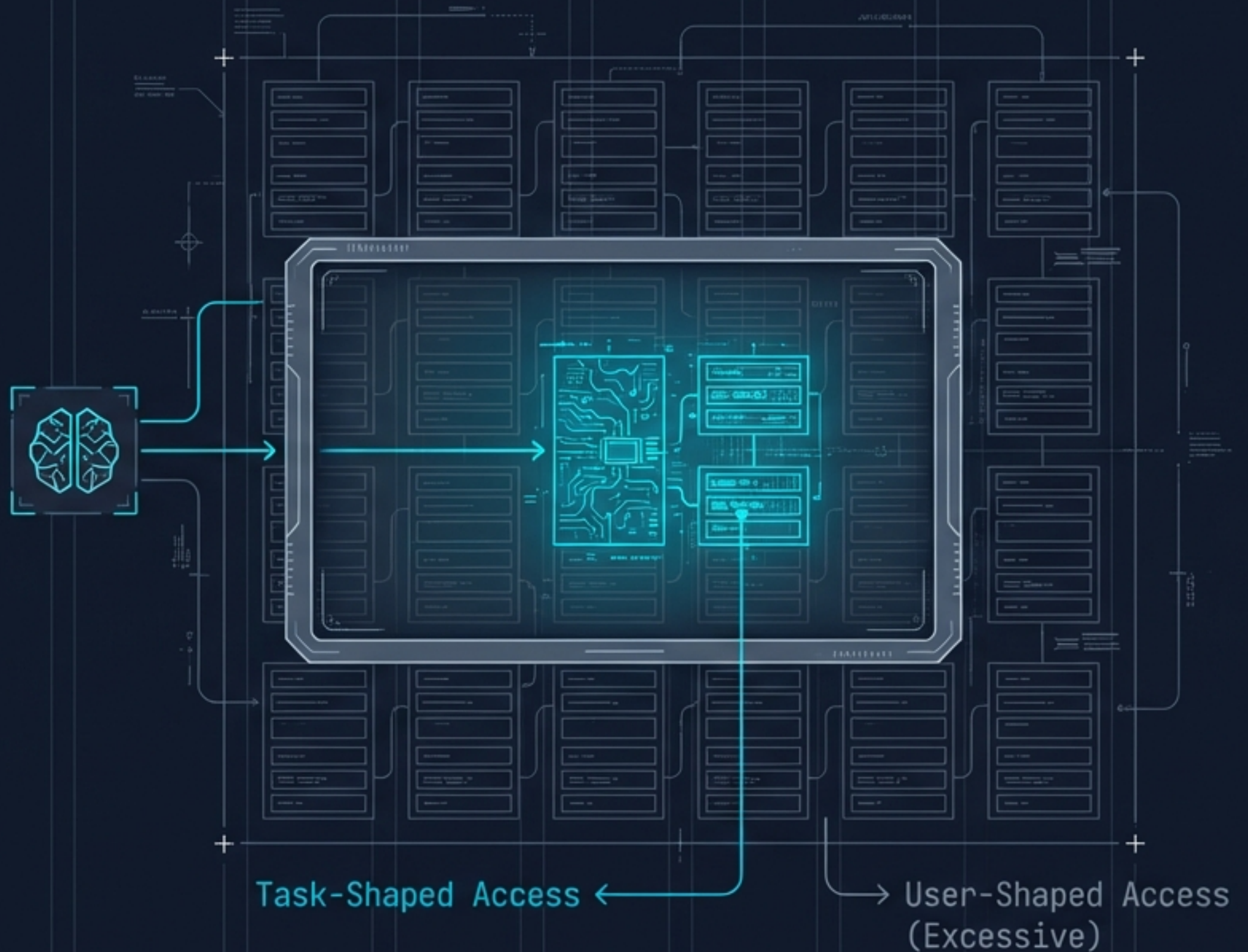
OWASP LLM06:
EXCESSIVE AGENCY

LEGACY AUTHORISATION IS TOO COARSE FOR AUTONOMOUS AGENTS

The Concept of Granularity Repair

Current state: "Read my email"
grants access to the entire inbox
history. This is Excessive Agency.

MAL Approach: "Summarise today's
email" restricts the view to
today's messages only. This is
Task-Shaped Access.



THE MODEL ABSTRACTION LAYER (MAL): A POLICY-DEFINED FAÇADE

Positioning the MAL within the stack.

The MAL looks and feels like the underlying resource to the agent, but strictly enforces policy boundaries behind the scenes.

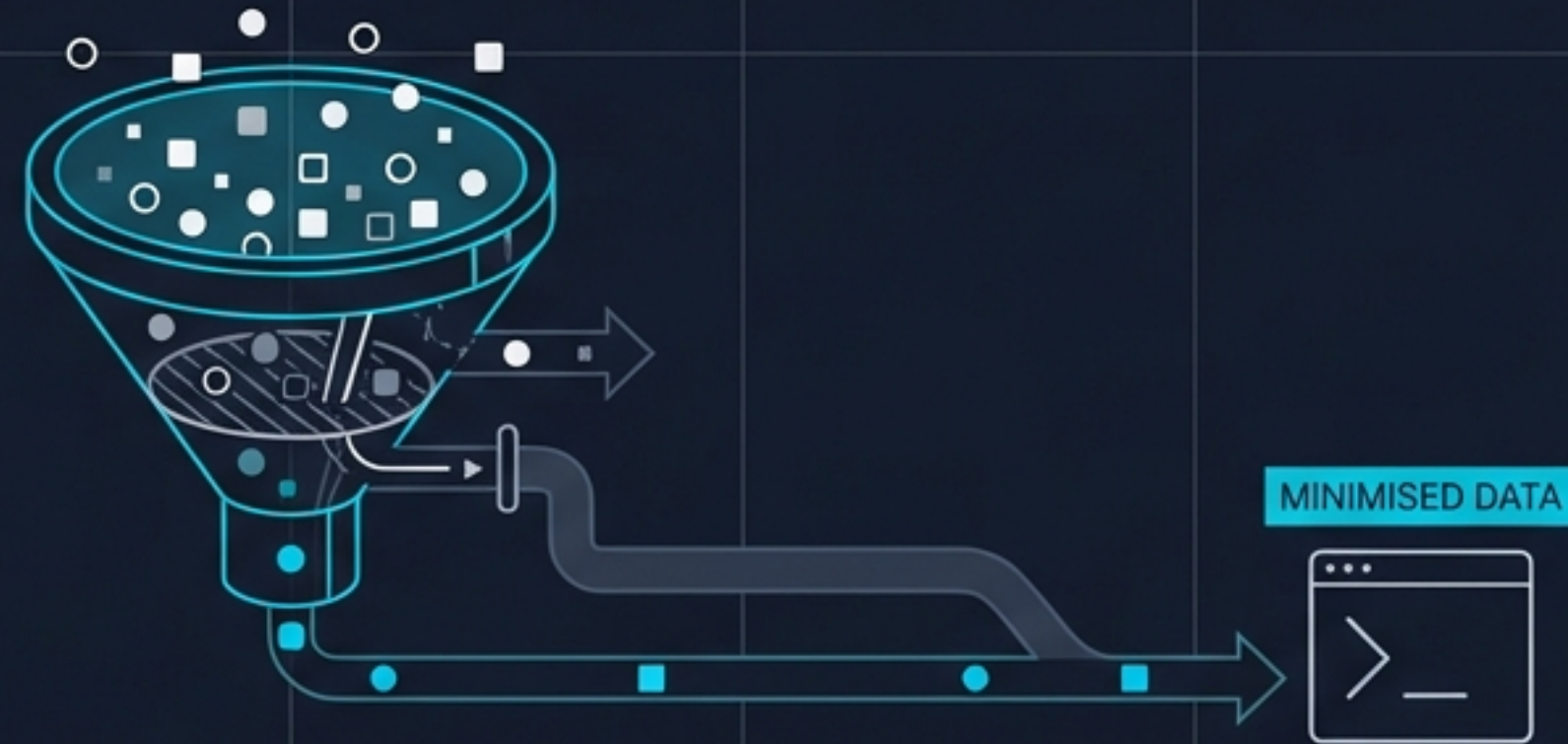


Two Planes of Protection: Constraining Data and Capability

Principle: Data Minimisation. Collect and share only what is adequate and relevant.

1. The Data Plane (Read Protection)

Principle: Data Minimisation. Collect and share only what is adequate and relevant.



2. The Capability Plane (Write Protection)

Principle: Least Privilege. Read-only by default. Step-up auth for writes.

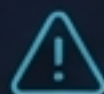


Operating Modes: Transparent Mimicry vs. Safety Normalisation

Transparent Mode (Raw / Dangerous)

```
{
  "id": "msg_12345",
  "timestamp": "2024-05-20T10:00:00Z",
  "headers": { ... },
  "body_html": "<div onmouseover=
'alert(1)'>Click for prize! <img src=x
onerror=alert('XSS')></div>",
  "metadata": { "status": "unread" }
}
```

Mimics real system.
High injection risk.



Normalised Mode (Structured / Safe)

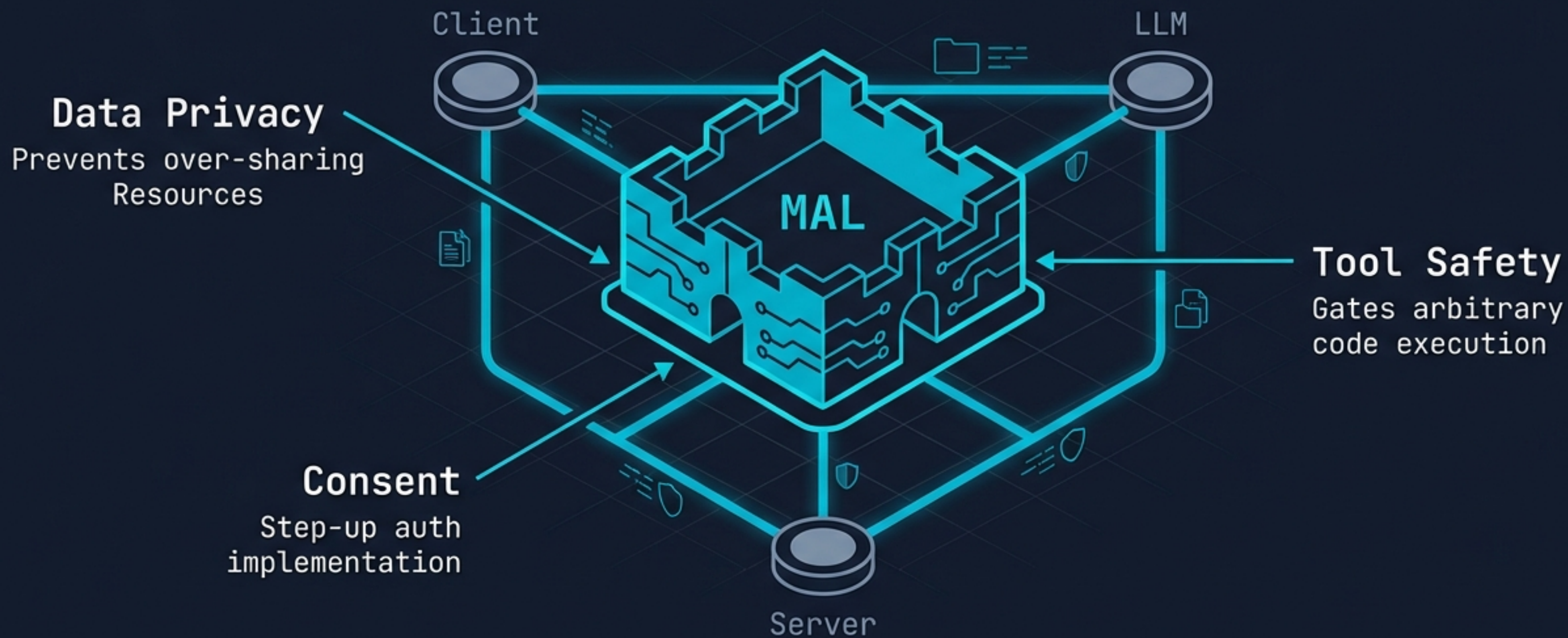
```
{
  "sender": "alice@example.com",
  "subject": "Project Update",
  "summary": "Meeting confirmed."
}
```

Transforms artifacts.
Removes injection vectors.

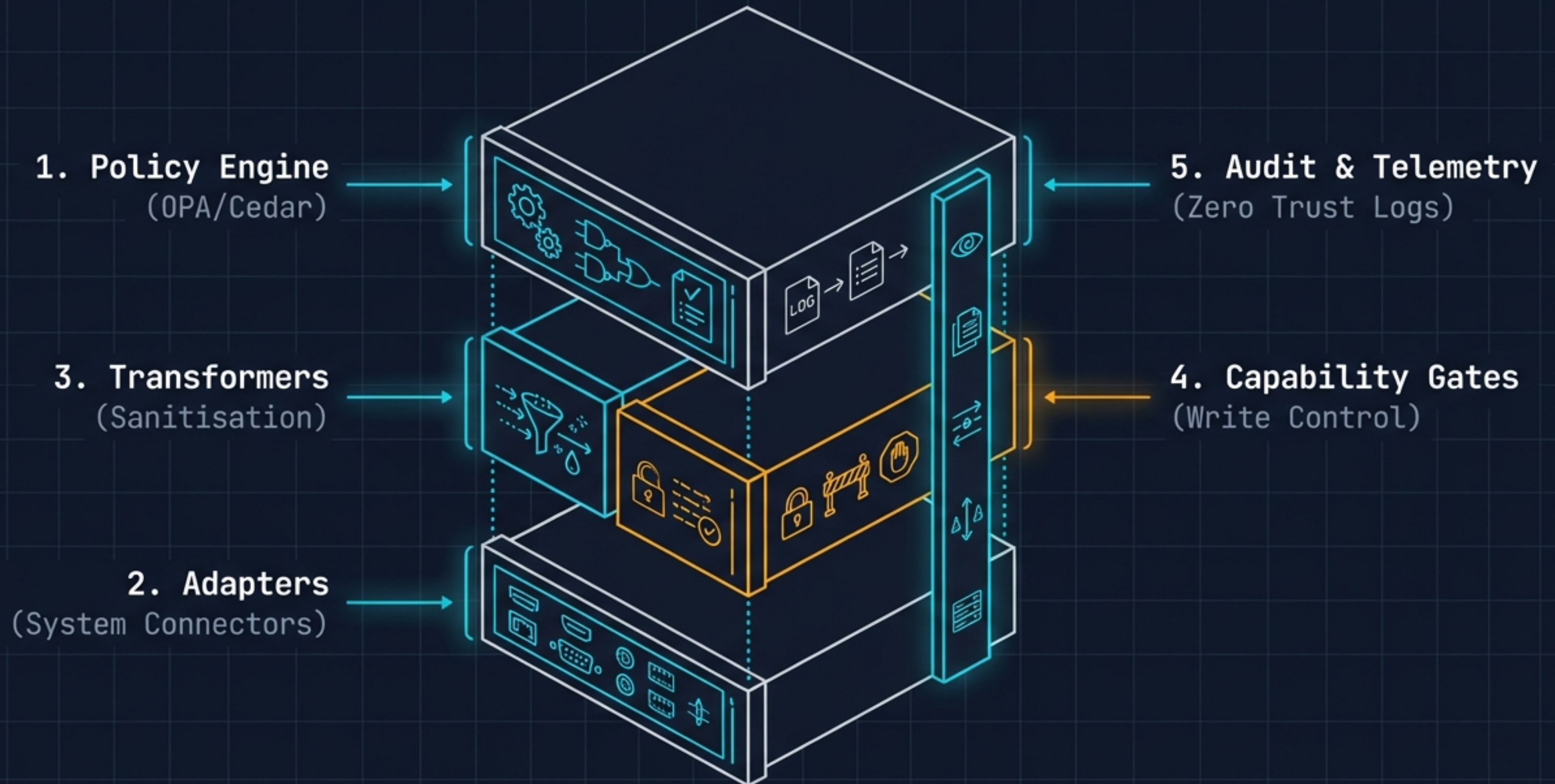
Structured outputs constrain data flow, eliminating hidden channels for prompt injection.

MAL Enforces the Security Principles of the Model Context Protocol

Protocol Compliance Strategy

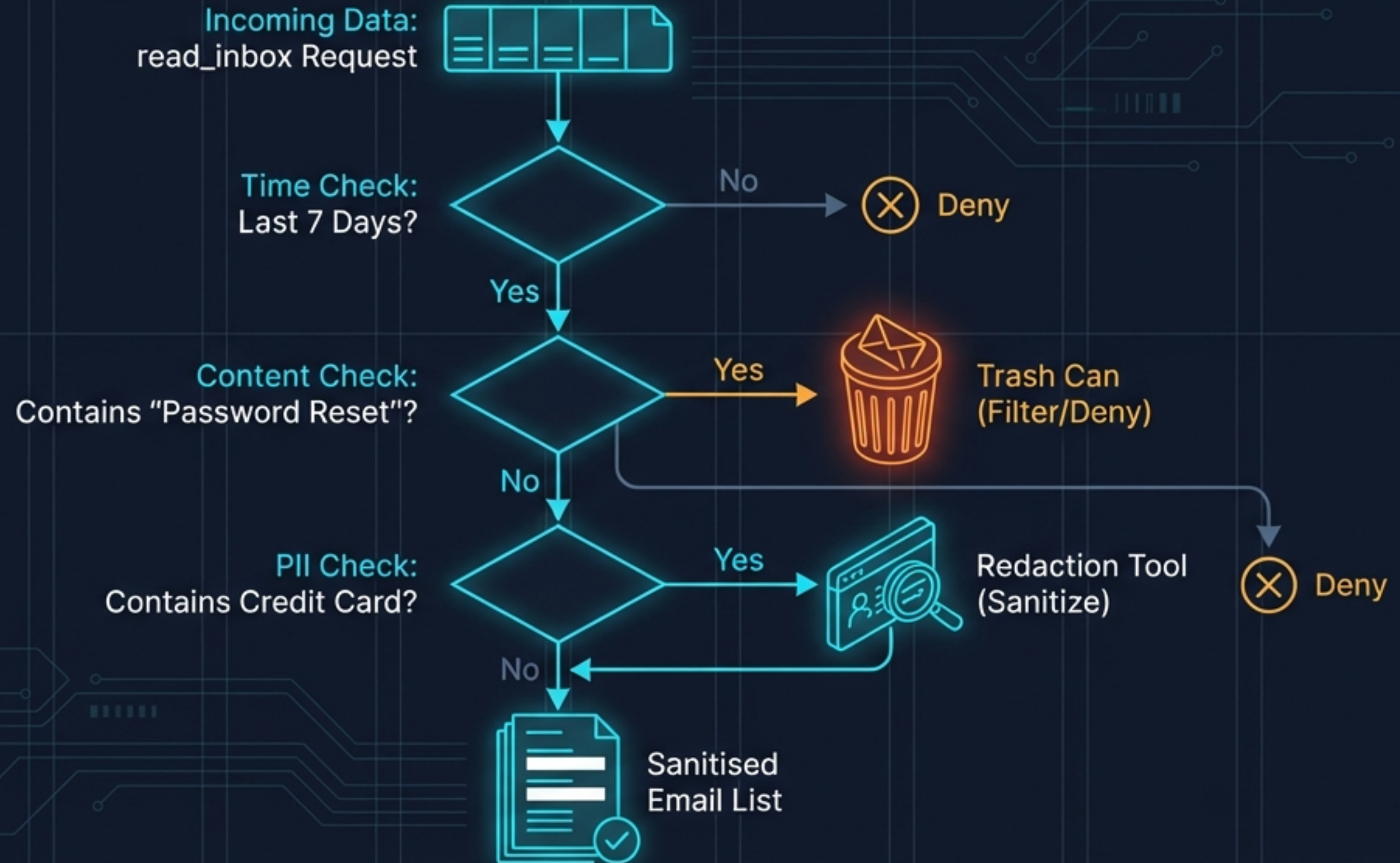


Anatomy of a MAL Implementation



Deep Dive: The 'Safe Inbox' Policy in Action

Scenario: Request: read_inbox



Capability Attenuation & Step-Up Authorisation

Attenuated Delegation using Macaroons



Green: Read-Only

Agent accesses
normalised data
autonomously.



Yellow: Draft/Verify

Agent drafts content.
Sending is blocked.



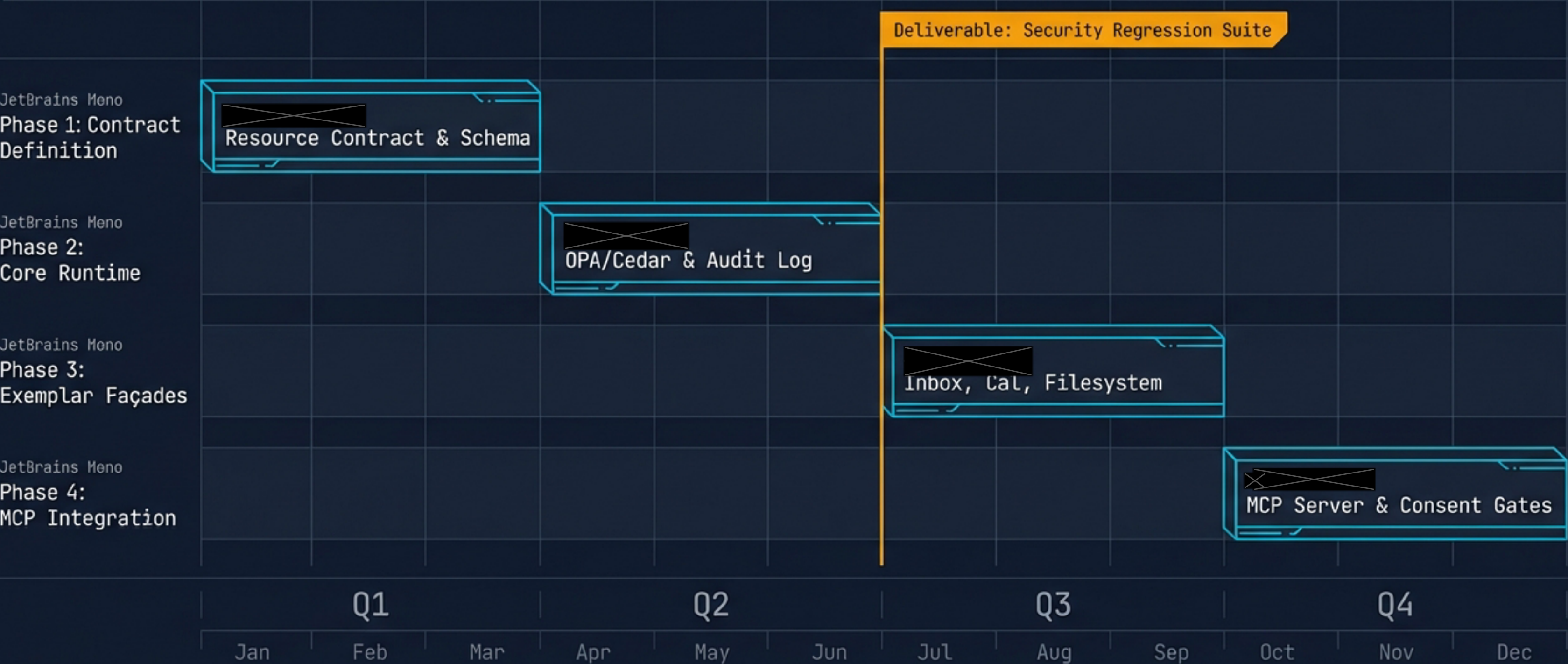
The Build Plan: A Multi-Agent Implementation Strategy

Strategy: Leveraging OpenAI Codex and Claude Code.



Development Roadmap: A Test-First Delivery Sequence

Timeline: Quarterly Delivery Sequence & Milestones



Building Community Credibility through Transparency

Validation Pillars



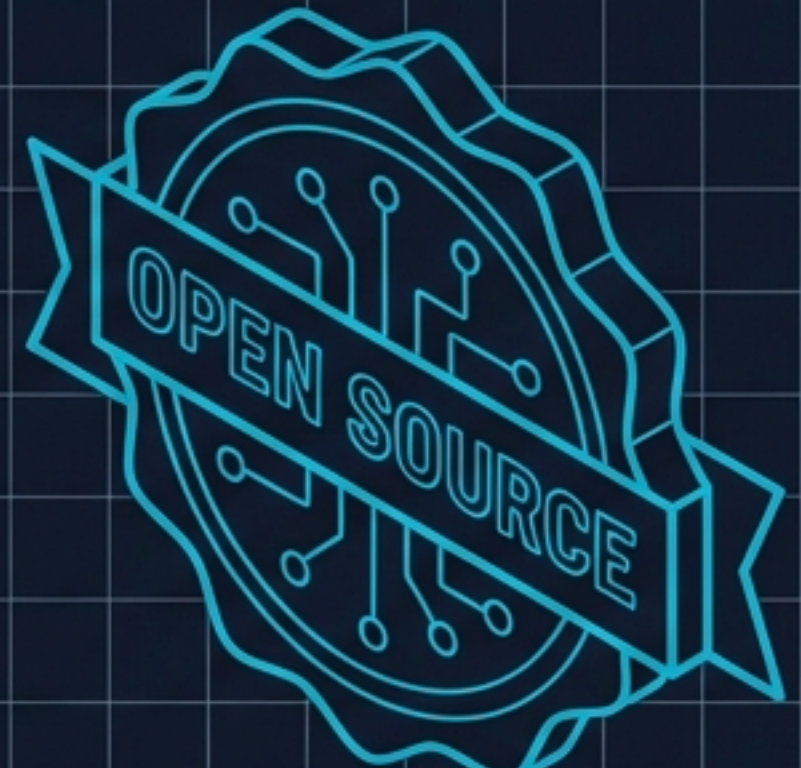
Threat Modeling

Mapped to MITRE ATLAS
Adversarial Landscape.



Shift-Left Testing

Automated Prompt
Injection Mitigation.



Open Source

Reference Implementation
& Known-Bad Test Corpus.

The Path Forward: Containment by Default

MCP standardises connection. MAL standardises safety.

Move from broad access to policy-defined abstractions.



Don't just connect agents. Engineer the boundaries that make them safe to deploy.